Лабораторная работа № 19: Исследование методов работы модуля Events.

Цель работы: Сформировать умение анализировать методы работы модуля Events.

Задание 1 (Базовый уровень)

Создание простого EventEmitter и подписка на события

Описание:
Необходимо создать базовый скрипт, демонстрирующий паттерн "Наблюдатель" с использованием класса `EventEmitter` из модуля `events`.

Как это должно работать:

1. Создаётся экземпляр `EventEmitter`
2. Регистрируются слушатели на несколько событий:
    - `greet` — выводит приветствие с переданным именем
    - `info` — выводит информацию о группе ББМО-01-23
    - `bye` — выводит прощание
3. События эмитятся по очереди
4. Демонстрируется возвращаемое значение `emit()`:
    - `true` — если были слушатели
    - `false` — если слушателей не было

Пример работы:

```
text
```

```
$ node events-basic.js

=== Демонстрация EventEmitter ===

[greet] Привет, Иван!
[info] Группа: ББМО-01-23
[bye] До свидания!

Событие "unknown" без слушателей: false

Событие "greet" со слушателями: true
```

Что нужно сделать для отчёта:

- Сделать скриншот запуска скрипта
- Сделать скриншот вывода всех событий
- Сделать скриншот возвращаемых значений `emit()`

Задание 2 (Средний уровень)

Методы `on`, `once`, `addListener` и управление подписками

Описание:
Изучить различные методы подписки на события и управления слушателями.

Как это должно работать:

1. Разница между `on` и `once`:
    - `on` — вызывается при каждом событии
    - `once` — вызывается только один раз
2. Демонстрация:
    - Подписка через `on` на событие `tick` — вызывается 3 раза
    - Подписка через `once` на событие `tick` — вызывается 1 раз
    - Сравнение в выводе
3. Управление подписками:
    - `listenerCount(eventName)` — сколько слушателей
    - `listeners(eventName)` — массив слушателей
    - `removeListener(eventName, listener)` — удаление конкретного
    - `removeAllListeners(eventName)` — удаление всех
4. Демонстрация порядка вызова:
    - Слушатели вызываются в порядке добавления
    - Для группы ББМО-01-23 добавить слушатель с номером группы

Пример работы:

```
text

$ node events-once.js

=== Сравнение on и once ===
[tick #1] on-слушатель
[tick #1] once-слушатель
[tick #2] on-слушатель
[tick #3] on-слушатель

on-слушатель вызван: 3 раза
once-слушатель вызван: 1 раз

=== Управление подписками ===
Слушателей до удаления: 3
Слушателей после удаления одного: 2
Слушателей после removeAllListeners: 0

=== Порядок вызова ===
```

```
1. Первый слушатель (группа ББМО-01-23)
2. Второй слушатель

3. Третий слушатель
```

Что нужно сделать для отчёта:

- Скриншот сравнения `on` и `once`
- Скриншот `listenerCount`
- Скриншот `removeListener` и `removeAllListeners`
- Скриншот порядка вызова слушателей

Задание 3 (Продвинутый уровень)

Наследование от EventEmitter

Описание:
Создать собственный класс, расширяющий `EventEmitter`, для моделирования работы базы данных или файловой системы.

Как это должно работать:

1. Класс `DatabaseConnection extends EventEmitter`:
    - Метод `connect()` — эмитит событие `connect` при успехе
    - Метод `query(sql)` — эмитит `query` с SQL-запросом и результатом
    - Метод `close()` — эмитит `close`
    - Метод `error()` — эмитит `error`
2. Демонстрация:
    - Создание экземпляра
    - Подписка на все события
    - Выполнение последовательности операций
    - Логирование каждого события
3. Паттерн "Ошибка как событие":
    - Событие `error` имеет специальный статус
    - Если нет слушателя `error` — Node.js выбросит исключение
    - Демонстрация корректной обработки
4. Для группы ББМО-01-23:
    - В сообщениях указывать номер группы

Пример работы:

```
text

$ node events-class.js
```

```
=== DatabaseConnection (группа ББМО-01-23) ===
[EVENT] Подключение к БД...
[EVENT] Соединение установлено
[EVENT] Выполнение запроса: SELECT * FROM students
[EVENT] Результат: 50 записей
[EVENT] Выполнение запроса: INSERT INTO students
[EVENT] Результат: 1 запись добавлена
[EVENT] Закрытие соединения
[EVENT] Соединение закрыто

=== Демонстрация ошибки ===
[EVENT] Ошибка: Connection timeout
```

(без слушателя error — процесс упал бы с исключением)

Что нужно сделать для отчёта:

- Скриншот класса `DatabaseConnection`
- Скриншот работы методов `connect`, `query`, `close`
- Скриншот обработки события `error`
- Скриншот демонстрации падения без слушателя `error`

Задание 4 (Сложный уровень)

Продвинутые концепции: `newListener`, `removeListener`, метапрограммирование

Описание:
Изучить специальные встроенные события `newListener` и `removeListener`, а также метапрограммирование через них.

Как это должно работать:

1. Событие `newListener`:
    - Срабатывает при добавлении любого слушателя
    - Аргументы: имя события, функция-слушатель
    - Используется для логирования и создания систем плагинов
2. Событие `removeListener`:
    - Срабатывает при удалении слушателя
    - Аргументы: имя события, функция-слушатель
    - Используется для очистки ресурсов
3. Практический пример — система плагинов:
    - Класс `PluginManager extends EventEmitter`
    - При регистрации плагина эмитится `plugin:registered`
    - `newListener` логирует все подписки

    - `removeListener` логирует все отписки
4. Демонстрация утечки памяти:
    - Создание множества слушателей без удаления
    - Показать рост `listenerCount`
    - Показать, как `removeListener` решает проблему
5. Асинхронные слушатели:
    - Слушатели могут быть async-функциями
    - `emit` не ждёт их выполнения
    - Использование `setImmediate` для отложенного выполнения
    - Для группы ББМО-01-23 — добавить префикс `[ВВМО-01-23]` в логи

Пример работы:

```
text
```

```
$ node events-advanced.js

=== Система плагинов ===
[newListener] Добавлен слушатель "plugin:registered"
[newListener] Добавлен слушатель "plugin:removed"

[PLUGIN] Зарегистрирован: LoggerPlugin
[newListener] Добавлен слушатель "log"

[PLUGIN] Зарегистрирован: AlertPlugin
[newListener] Добавлен слушатель "alert"

[removeListener] Удалён слушатель "log"
[PLUGIN] Удалён: LoggerPlugin

=== Демонстрация утечки ===
Слушателей до: 10
Слушателей после добавления 1000: 1010
Слушателей после removeAllListeners: 0

=== Асинхронные слушатели ===
[ВВМО-01-23] emit вызван (не ждёт async-слушатель)
```

```
[ВВМО-01-23] async-слушатель завершён через 100ms
```

Что нужно сделать для отчёта:

- Скриншот работы `newListener`
- Скриншот работы `removeListener`
- Скриншот системы плагинов
- Скриншот демонстрации утечки памяти
- Скриншот асинхронных слушателей

Задание 5 (Экспертный уровень)

Полноценная система событий: EventBus с приоритетами, once-кэшем и wildcard-событиями

Описание:
Создать полнофункциональную систему событий (EventBus), расширяющую `EventEmitter`, с поддержкой приоритетов, wildcard-событий, once-кэша и метрик.

Как это должно работать:

1. Класс `EventBus extends EventEmitter`:
    - `on(event, listener, priority)` — подписка с приоритетом (чем выше, тем раньше вызовется)
    - `once(event, listener, priority)` — одноразовая подписка
    - `emit(event, ...args)` — вызов с учётом приоритетов
    - `onAny(listener)` — подписка на все события (wildcard)
    - `off(event, listener)` — отписка
2. Приоритеты:
    - Слушатели сортируются по приоритету (по убыванию)
    - Приоритет по умолчанию — 0
    - Слушатели с одинаковым приоритетом вызываются в порядке добавления
3. Wildcard-события:
    - Слушатель `onAny` вызывается при любом событии
    - Получает имя события и аргументы
    - Используется для логирования и мониторинга
4. Once-кэш:
    - `once` автоматически удаляется после первого вызова
    - Демонстрация: `once`-слушатель вызывается ровно один раз
5. Метрики:
    - Счётчик вызовов каждого события
    - Общее количество событий
    - Время последнего вызова
    - Метод `getMetrics()`
6. Обработка ошибок:
    - Слушатель `error` с высоким приоритетом
    - Все ошибки логируются с указанием группы ББМО-01-23
    - Демонстрация обработки ошибок в слушателях (try/catch)
7. Интеграция с модулями `os` и `http`:
    - EventBus используется для событий мониторинга системы
    - HTTP-сервер эмитит событие `request` при каждом запросе
    - `GET /metrics` возвращает метрики EventBus

Пример работы:

```
text
```

```
$ node event-bus.js

=== EventBus (группа ББМО-01-23) ===

--- Приоритеты ---
[priority 10] Высокий приоритет
[priority 5]  Средний приоритет
[priority 0]  Низкий приоритет

--- Wildcard ---
[onAny] Событие "greet" с аргументами: ["Иван"]
[onAny] Событие "info" с аргументами: ["ББМО-01-23"]

--- Once-кэш ---
[tick] Вызов 1
[tick] Вызов 2
[tick] Вызов 3
once-слушатель вызван: 1 раз

--- Обработка ошибок ---
[ERROR] [ББМО-01-23] Ошибка в слушателе "test": ...
[ERROR] [ББМО-01-23] Ошибка в слушателе "test2": ...

--- Метрики ---
greet:  2 вызова
info:   1 вызов
tick:   3 вызова
test:   2 вызова (1 ошибка)
Всего событий: 8

$ curl http://localhost:3000/metrics
{
  "group": "ББМО-01-23",
  "events": {
    "greet": 2,
    "info": 1,
    "tick": 3,
    "request": 5
  },
  "total": 11,
  "listeners": {
    "greet": 1,
    "tick": 1
  }
}
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта
- Скриншот работы приоритетов
- Скриншот работы wildcard-событий
- Скриншот once-кэша
- Скриншот обработки ошибок
- Скриншот метрик
- Скриншот HTTP-запроса `GET /metrics`
- Скриншот интеграции с `os` и `http`

Фиксация изменений

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request